

`device\_map="auto"` vs On-Demand Layering 비교 분석

날짜

2026-03-09

작성자

노승우

Alpamayo VRAM Optimization Research

목차 (Table of Contents)

1. 실험 개요
2. Device Map 분배 결과
3. 추론 시간 비교
4. 핵심 발견
 - 5070 Ti 역설 — 더 빠른 GPU, 더 느린 추론
 - device_map="auto"의 구조적 한계
 - On-Demand Layering의 차별점
5. device_map="auto" 동작 방식 상세
6. 결론
- 참고

`device_map="auto"` vs On-Demand Layering 비교 분석

작성일: 2026-03-09 환경: RTX 3080 Ti (12GB VRAM), RTX 5070 Ti (16GB VRAM) 대상
모델: Alpamayo-R1-10B (총 VRAM 요구 21.52GB FP16)

1. 실험 개요

Alpamayo-R1-10B 모델에 HuggingFace accelerate의 `device_map="auto"` 방식 적용 테스트.
RTX 3080 Ti (12GB)와 RTX 5070 Ti (16GB) 두 환경에서 레이어 분배 결과 및 추론 시간 측정.
On-Demand Layering (D2H 제거, 실험 `01-on-demand-layering`) 결과와 대조 분석.

2. Device Map 분배 결과

accelerate의 `infer_auto_device_map()` 이 각 환경의 `max_memory` 설정에 따라 모듈을 GPU 또는 CPU에 자동 배치한 결과.

컴포넌트	파라미터	3080 Ti (12GB)	5070 Ti (16GB)
Vision Encoder	~0.68B	GPU	GPU
Embed Tokens	~0.05B	GPU	GPU
VLM Layer 0-16 (17개)	~6.55B	GPU	GPU
VLM Layer 17-27 (11개)	~4.24B	CPU	GPU
VLM Layer 28-35 (8개)	~3.08B	CPU	CPU
norm, rotary_emb	~0.01B	CPU	CPU
lm_head	~0.03B	CPU	CPU
expert	~2.28B	CPU	CPU
action_space, diffusion	—	CPU	CPU
action_in/out_proj	—	CPU	CPU

3080 Ti: GPU 할당 ~7.28B (17L + Vision + Embed), CPU 할당 ~9.63B **5070 Ti:** GPU 할당 ~11.52B (28L + Vision + Embed), CPU 할당 ~5.39B

3. 추론 시간 비교

환경	GPU 레이어	CPU 레이어	추론 시간	대비
3080 Ti Unified Memory (베이스 라인)	전체 (CUDA 스왑)	—	273.79s	1x
5070 Ti <code>device_map="auto"</code>	VLM 28L + 기타	VLM 8L + expert 등	176.04s	1.56x
3080 Ti <code>device_map="auto"</code>	VLM 17L + 기타	VLM 19L + expert 등	57.75s	4.74x
3080 Ti On-Demand Layering	6L 상주 + 30L 동적	30L 저장	43.38s	6.31x

4. 핵심 발견

5070 Ti 역설 — 더 빠른 GPU, 더 느린 추론

5070 Ti는 3080 Ti보다 GPU에 레이어를 11개 더 배치했음에도 추론 시간이 약 3배 느림 (176.04s vs 57.75s).

원인 분석: - `expert` 모듈 (~2.28B 파라미터)과 `diffusion` 모듈이 두 환경 모두 CPU 배치 → CPU 연산 병목 발생 - `expert` 모듈은 VLM 이후 실행되는 핵심 경로임에도 파라미터 크기로 인해 GPU 배치에서 제외 - 5070 Ti의 넓은 VRAM이 `expert` 배치에 활용되지 못한 채 낭비됨 - VLM 레이어 수 증가보다 `expert` / `diffusion` CPU 병목이 전체 지연을 지배

`device_map="auto"`의 구조적 한계

- `infer_auto_device_map()`은 모듈별 파라미터 크기 합산 기준으로 순차 배치
- 커스텀 모듈(`expert`, `diffusion`, `action_space`)의 실행 특성(연산 집약도, 호출 빈도)을 고려하지 않음
- CPU 배치된 모듈에는 `AlignDevicesHook`이 삽입되어 입력 텐서를 CPU로 자동 이동 후 CPU에서 연산 수행
- 모델 구조 전체를 파악하지 못한 자동 배치로 핵심 병목 모듈이 CPU에 잔류

On-Demand Layering의 차별점

항목	<code>device_map="auto"</code>	On-Demand Layering
배치 기준	파라미터 크기 (자동)	실행 순서 + 수동 제어
연산 위치	GPU + CPU 혼재	전체 GPU
CPU 역할	실시간 연산 참여	웨이트 저장만
<code>expert</code> / <code>diffusion</code>	CPU 연산	GPU 연산
제어 복잡도	낮음	높음

On-Demand Layering은 모든 forward pass를 GPU에서 수행하고 CPU는 웨이트 저장소로만 활용. `expert`와 `diffusion`을 포함한 전 모듈을 GPU 연산으로 처리하므로 근본적으로 다른 접근.

5. `device_map="auto"` 동작 방식 상세

[모델 로딩 시]

1. `accelerate.infer_auto_device_map()` 호출
 - 모듈 트리 순회, 각 모듈의 파라미터 크기 계산
 - `max_memory={'cuda:0': '11GiB', 'cpu': '14GiB'}` 기준
 - 순서대로 GPU에 채움 → 초과분을 CPU로
2. `AlignDevicesHook` 삽입 (CPU 모듈에)
 - `forward()` 호출 전 입력 텐서를 해당 디바이스로 이동
 - `forward()` 완료 후 출력 텐서를 이전 디바이스로 반환

[추론 시]

GPU 모듈 → (자동 텐서 이동) → CPU 모듈 → (자동 텐서 이동) → ...

이전 실험(01-on-demand-layering)과의 차이: 과거 테스트에서는 `device_map="auto"` 적용 시 오류 발생. 현재 성공 → `accelerate` 또는 `transformers` 버전 업데이트로 커스텀 아키텍처 지원 개선된 것으로 추정.

6. 결론

`device_map="auto"` 평가: - 설정 간편, 코드 수정 최소화 - 커스텀 아키텍처(expert, diffusion)에 최적 배치 - CPU 연산 병목 해소 불가 - 더 큰 VRAM 환경에서도 효과 제한적 (5070 Ti 사례)

On-Demand Layering 우위: - 수동 제어로 전 연산을 GPU 수행 - 현재 최고 성과: **43.38s** (6.31x 개선) - CPU 병목 구조적 해소

5070 Ti 대응: - `device_map="auto"` 적용 효과 미미 (1.56x에 그침) - On-Demand Layering 이식 시 16GB VRAM 활용으로 추가 개선 기대 - 상주 레이어 수 증가 가능 → D2H 교환 횟수 감소 예상

참고

- 관련 실험: [research/01-on-demand-layering/](#) (On-Demand Layering 구현 및 결과)
- 관련 실험: [research/05-vram-limit-comparison/](#) (3080 Ti vs 5070 Ti 비교)
- accelerate 문서: [infer_auto_device_map](#), [AlignDevicesHook](#)